

Binary Search Tree – README

Overview

This code implements a simple Binary Search Tree (BST) for storing and managing a phone directory. BST allows you to insert, search, delete entries, and print the phone directory in lexicographically sorted order by names. Each node in the tree stores a contact's name as the key and their phone number as the associated value.

Features

1. Insert: Add a new contact with a name and phone number.
2. Search: Look up a contact's phone number using their name.
3. In-order Traversal: Print all contacts in lexicographical order by name.
4. Delete: Remove a contact by name from the directory.

Classes

1. TreeNode

Represents a node in the binary search tree.

- Attributes:
 - name (str): The key of the node, representing the contact's name.
 - phone_number (str): The value of the node, representing the contact's phone number.
 - left (TreeNode): Pointer to the left child (names lexicographically smaller).
 - right (TreeNode): Pointer to the right child (names lexicographically larger).

2. BinarySearchTree

Manages the operations on the BST, such as inserting, searching, deleting, and traversing nodes.

Key Methods:

- insert (name, phone_number): Inserts a new node with the given name and phone number. If the tree is empty, the node becomes the root. Otherwise, the appropriate position is determined using recursive comparisons.

- `search(name)`: Searches for a contact's phone number using their name. Returns `None` if the name is not found.
- `delete(name)`: Removes a node with the given name. Handles three cases:
 1. Leaf node (no children).
 2. Node with one child.
 3. Node with two children, where the in-order successor is used to maintain the BST structure.
- `inorder_traversal()`: Performs an in-order traversal of the tree, printing the names and phone numbers in lexicographically sorted order.

How to Run

1. Clone or download the repository containing the simple calculator script.
2. In Jupyter Notebook, simply paste the code into a cell and run the cell.